

# MCP Agora

Server MCP con memoria persistente cross-agente per flotte di agenti AI

Antonio Cioffi

2026

## Sommario

MCP Agora è un progetto portfolio/learning che implementa un MCP Server (Model Context Protocol) con memoria persistente cross-agente per flotte di agenti AI personali. Utilizza ChromaDB per indicizzazione vettoriale, sentence-transformers per embedding semantici (modello all-MiniLM-L6-v2, 384 dimensioni), SQLite per metadati e cache persistente, e FastMCP per l'esposizione di 8 tool MCP. L'architettura è organizzata in 7 layer (Transport, Protocol, Router, Memory, Cache, Backend Connector, Embedding) con routing semantico a backend MCP esterni (GitHub, Playwright). Il progetto conta 61 test automatici distribuiti su 10 file, zero mock, con ChromaDB e SQLite reali in directory isolate.

**Repository:** <https://github.com/cioffiAI/mcp-agora>

# Indice

---

|          |                                        |           |
|----------|----------------------------------------|-----------|
| <b>1</b> | <b>Introduzione e Obiettivi</b>        | <b>2</b>  |
| 1.1      | Obiettivi di Prodotto . . . . .        | 2         |
| 1.2      | Obiettivi di Apprendimento . . . . .   | 2         |
| 1.3      | Posizionamento . . . . .               | 2         |
| <b>2</b> | <b>Stack Tecnologico</b>               | <b>2</b>  |
| 2.1      | Scelte Tecnologiche . . . . .          | 3         |
| <b>3</b> | <b>Architettura</b>                    | <b>4</b>  |
| 3.1      | Diagramma dell'Architettura . . . . .  | 4         |
| 3.2      | Descrizione dei Layer . . . . .        | 6         |
| <b>4</b> | <b>API Tools</b>                       | <b>7</b>  |
| <b>5</b> | <b>Schema Dati</b>                     | <b>7</b>  |
| 5.1      | ChromaDB: Vector Store . . . . .       | 7         |
| 5.2      | SQLite: Database Relazionale . . . . . | 8         |
| <b>6</b> | <b>Testing</b>                         | <b>8</b>  |
| <b>7</b> | <b>Decisioni Progettuali</b>           | <b>9</b>  |
| <b>8</b> | <b>Conclusioni e Roadmap</b>           | <b>10</b> |
| 8.1      | Roadmap Completata . . . . .           | 10        |
| 8.2      | Roadmap Futura . . . . .               | 10        |
| 8.3      | Metriche di Successo . . . . .         | 10        |
| <b>A</b> | <b>config.yaml Completo</b>            | <b>11</b> |
| <b>B</b> | <b>Struttura del Progetto</b>          | <b>12</b> |
| <b>C</b> | <b>Cronologia Commit</b>               | <b>12</b> |

# 1 Introduzione e Obiettivi

---

MCP Agora è un progetto portfolio/learning che implementa un MCP Server (Model Context Protocol) con memoria persistente cross-agente per flotte di agenti AI personali. Nasce dall'esigenza reale di risolvere un problema concreto: gli agenti AI moderni (Claude Code, Codex CLI, ChatGPT, Gemini CLI) operano in sessioni isolate, senza memoria condivisa. Ogni agente ripete ricerche, ricontesta informazioni e duplica lavoro che un altro agente ha già fatto.

## 1.1 Obiettivi di Prodotto

- Un agente salva conoscenza → un altro agente la recupera (zero duplicazione).
- Una query già fatta → risposta cached (zero ricalcolo).
- Un errore già fatto → non si ripete (memoria delle decisioni).
- 4+ agenti condividono contesto senza configurazioni individuali.

## 1.2 Obiettivi di Apprendimento

- MCP Protocol deep dive: implementazione MCP Server compliant con SDK ufficiale.
- Vector search & embeddings: integrazione ChromaDB + sentence-transformers.
- System design a layer: architettura modulare con 7 layer separati e interfacce astratte.
- Async Python: asyncio per concorrenza su MCP transport.
- SQLite avanzato: schema relazionale per metadati, provenance, analytics.
- Caching strategies: TTLCache in-memory + SQLite persistente.

## 1.3 Posizionamento

Agora non compete con progetti enterprise come ContextForge (IBM), MetaMCP, AutoMem o mcp-memory-service. È un progetto portfolio che dimostra competenza su AI infrastructure, MCP protocol, vector search, semantic routing e system design, risolvendo un problema reale nei workflow personali con agenti. Il suo valore non è nell'originalità del concetto, ma nella qualità dell'esecuzione: stack 2026 (MCP, vettori, embedding), architettura pulita (7 layer, dipendenze astratte, testabile), e problematica reale (memoria isolata tra agenti).

# 2 Stack Tecnologico

---

Il progetto è interamente sviluppato in Python 3.13+ con uv come package manager. La tabella seguente riassume i componenti principali dello stack.

Tabella 1: Componenti principali dello stack tecnologico.

| Componente        | Tecnologia                  | Versione     |
|-------------------|-----------------------------|--------------|
| Runtime           | Python                      | $\geq 3.13$  |
| Package Manager   | uv                          | $\geq 0.11$  |
| MCP Framework     | FastMCP (mcp SDK)           | $\geq 1.0.0$ |
| Vector Store      | ChromaDB (PersistentClient) | latest       |
| Embeddings        | sentence-transformers       | $\geq 3.0.0$ |
| Modello Embedding | all-MiniLM-L6-v2            | 384d         |
| Cache L1          | cachetools TTLCache         | latest       |
| Cache L2 / DB     | SQLite3 (stdlib)            | built-in     |
| Config            | PyYAML                      | latest       |
| HTTP Client       | httpx                       | latest       |
| PDF Generation    | fpdf2                       | 2.8.7        |
| Testing           | pytest + pytest-asyncio     | $\geq 9.0.3$ |

## 2.1 Scelte Tecnologiche

### FastMCP

L'SDK ufficiale Anthropic offre FastMCP, un wrapper ad alto livello che semplifica la creazione di server MCP. Rispetto al low-level Server, richiede meno boilerplate e permette di focalizzarsi sulla logica applicativa.

### ChromaDB

Database vettoriale embedded, zero config, zero cloud. Supporta ANN index (HNSW) per ricerca in  $O(\log n)$  e cosine distance. La libreria PersistentClient garantisce persistenza su disco senza server esterno.

### sentence-transformers

Modello all-MiniLM-L6-v2: 384 dimensioni, circa 80MB, sufficiente per MVP. Viene caricato lazy (non all'import) e cachato in `~/ .cache/agora/models/`.

### SQLite

Per metadati, provenance e cache L2. Zero configurazione, file-based, incluso nella libreria standard di Python. Ogni operazione apre e chiude una connessione (thread-safe).

## **3 Architettura**

---

L'architettura di MCP Agora è organizzata in 7 layer separati, ciascuno con responsabilità ben definite e interfacce astratte. Questo approccio modulare permette di testare, sostituire e far evolvere ogni layer indipendentemente.

### **3.1 Diagramma dell'Architettura**

# MCP Agora - Architettura locale a memoria condivisa

Server MCP Python-only per memoria persistente cross-agente, cache, routing e connettori MCP esterni

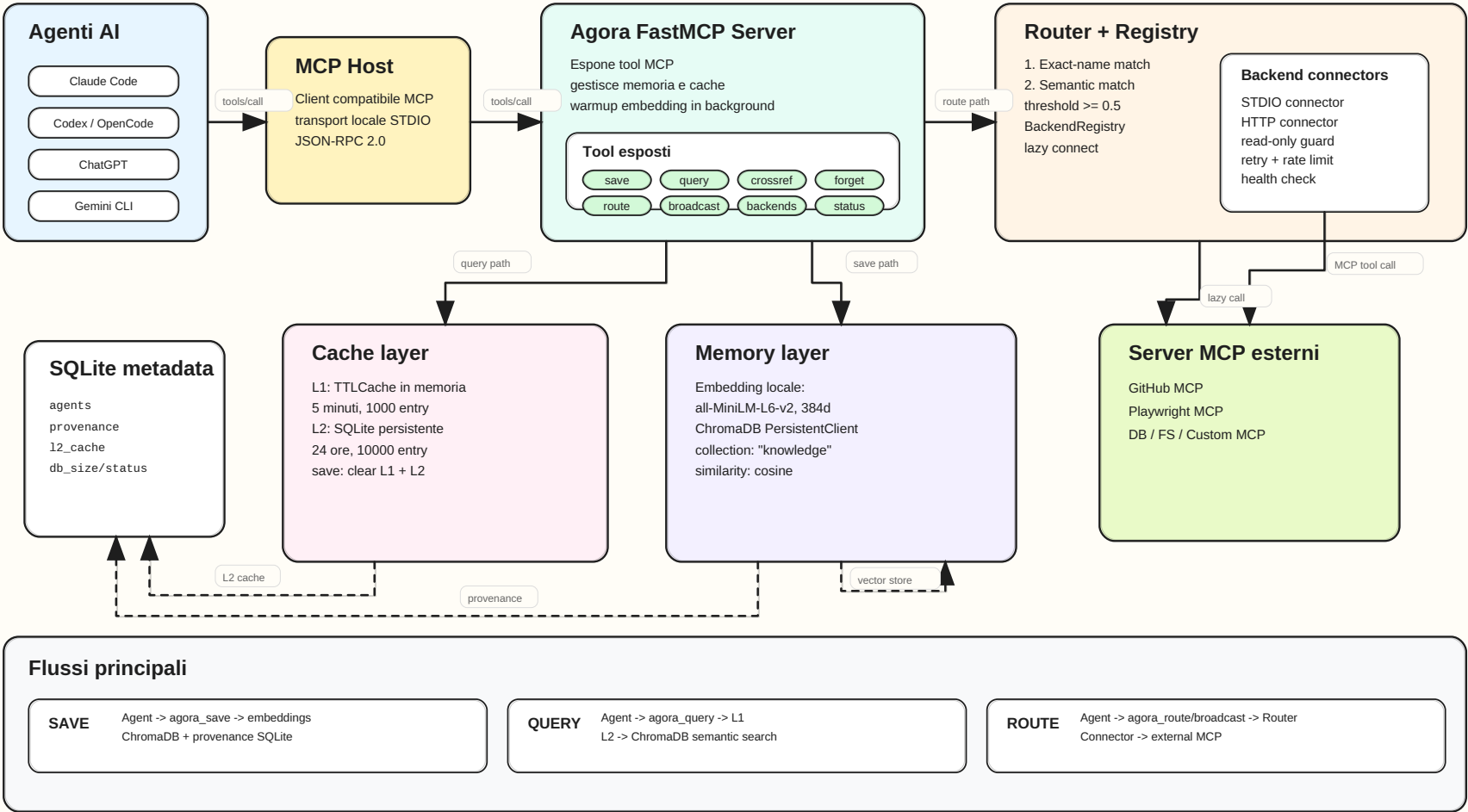


Figura 1: Architettura locale a memoria condivisa di MCP Agora.

## 3.2 Descrizione dei Layer

### Layer 1: Transport Layer

Gestisce la comunicazione con gli agenti via JSON-RPC 2.0. Supporta due trasporti: STDIO (locale, stdin/stdout, default) per agenti come Claude Code e Codex CLI; Streamable HTTP (remoto, HTTP POST + SSE) per agenti remoti o multi-sessione.

### Layer 2: Protocol Layer

Espone le primitive MCP standard: tools/list per scoprire i tool disponibili, tools/call per invocarli. Agora espone 8 tool (v. Sezione 4).

### Layer 3: Router Layer

Cuore decisionale del sistema. Quando un agente chiama `agora_route`, il router tenta prima un match esatto per nome (case-insensitive), poi un match semantico via cosine similarity sulle descrizioni embedding dei backend (soglia  $\geq 0.5$ ). Se nessun match, ritorna errore. Il routing broadcasting (`agora_broadcast`) invia la richiesta in parallelo a tutti i backend, limitato ai soli tool read-only.

### Layer 4: Memory Layer

Composto da due sotto-layer: Vector Index (ChromaDB, collezione knowledge, spazio coseno, 384 dimensioni) per ricerca semantica, e Relational DB (SQLite3, tabelle agents, provenance, l2\_cache) per metadati strutturati. La retention della knowledge è permanente (finché non cancellata con `agora_forget`). La cache L2 ha TTL configurabile (default 24h).

### Layer 5: Cache Layer

Due livelli di cache in cascata. L1: cachetools TTLCache in-memory (1000 entry, 5 minuti TTL). L2: SQLite-backed su disco (10000 entry, 24h TTL). Cache key = SHA256 della richiesta completa JSON serializzata con `sort_keys=True`. Su `agora_save` e `agora_forget`, entrambe le cache vengono invalidate (clear totale). Cascade: L1 → L2 → ChromaDB. Su miss ChromaDB, entrambe le cache vengono populate.

### Layer 6: Backend Connector Layer

Ogni backend MCP esterno è rappresentato da un connettore che implementa l'interfaccia astratta BackendConnector. Due implementazioni: StdioConnector (subprocess MCP via `stdio_client` + ClientSession, lazy connect, `asyncio.timeout`) e HttpConnector (streamable HTTP MCP via `streamablehttp_client`). La connessione è lazy: il connettore si connette al primo uso, non all'avvio. Read-only enforcement: se un backend è marcato `read_only`, solo i tool con prefissi whitelist (`list_`, `get_`, `fetch_`, `read_`, `search_`, `find_`, `query_`, `describe_`, `show_`) possono essere chiamati. I tool mutativi bloccati sollevano `ReadOnlyBlockedError`.

## Layer 7: Embedding Layer

Provider astratto `EmbeddingProvider` con implementazione concreta `SentenceTransformerProvider` che utilizza `all-MiniLM-L6-v2` (384 dimensioni). Lazy loading: il modello viene caricato al primo uso, non all'import del modulo. Cache in `~/ .cache/agora/models/`. Prima chiamata circa 17s (download modello + torch). Chiamate successive <1s. Il metodo `warmup()` permette di pre-caricare il modello all'avvio del server.

## 4 API Tools

Agora espone 8 tool MCP, registrati via decoratore `@mcp.tool()` su `FastMCP`. Tutti i tool accettano e restituiscono dizionari JSON serializzabili.

Tabella 2: Tool MCP esposti da Agora.

| Tool                         | Parametri                                                                                                      | Comportamento                                                                                                                   | Cache                |
|------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|----------------------|
| <code>agora_save</code>      | <code>content</code> , <code>tags</code> , <code>agent</code> , <code>session</code> , <code>confidence</code> | Salva conoscenza in ChromaDB + provenance in SQLite. Registra agente.                                                           | Clear L1+L2          |
| <code>agora_query</code>     | <code>query</code> , <code>top_k</code> (=5)                                                                   | Ricerca semantica. Cascade: L1 → L2 → ChromaDB.                                                                                 | Popola L1+L2 su miss |
| <code>agora_route</code>     | <code>target</code> , <code>arguments</code>                                                                   | Routing a backend MCP. Exact match → semantic ( $\geq 0.5$ ).                                                                   | No                   |
| <code>agora_broadcast</code> | <code>tool</code> , <code>arguments</code>                                                                     | Broadcast parallelo (asincio.gather) a tutti i backend. Solo read-only.                                                         | No                   |
| <code>agora_backends</code>  | (none)                                                                                                         | Lista backend configurati + stato connessione.                                                                                  | No                   |
| <code>agora_crossref</code>  | <code>query</code> , <code>entry_id</code> , <code>top_k</code>                                                | Correlazioni cross-agente. Per query: raggruppa per agente. Per ID: trova entry correlate.                                      | No                   |
| <code>agora_forget</code>    | <code>entry_ids</code> , <code>tags</code> , <code>agent</code> , <code>dry_run</code>                         | Elimina memoria. Filtri composti. <code>dry_run</code> preview. Clear L1+L2.                                                    | Clear L1+L2          |
| <code>agora_status</code>    | (none)                                                                                                         | Statistiche: <code>entry</code> , <code>agenti</code> , <code>cache</code> L1+L2, <code>backend</code> , <code>db_size</code> . | No                   |

## 5 Schema Dati

### 5.1 ChromaDB: Vector Store

Collezione unica knowledge con spazio metrico coseno. I documenti sono i testi salvati, gli embedding sono vettori a 384 dimensioni generati da `all-MiniLM-L6-v2`, e i metadati includono `tags` (formato CSV), `created_at` (ISO 8601) e `agent`.



Il wrapper VectorStore (agora/memory/vector\_store.py) astrae l'API ChromaDB e gestisce automaticamente la creazione della collezione e la serializzazione.

## 5.2 SQLite: Database Relazionale

Tre tabelle per metadati strutturati: agents (registry agenti), provenance (tracciabilità delle entry di conoscenza), l2\_cache (cache persistente su disco).

Listing 1: Schema relazionale SQLite.

```
CREATE TABLE IF NOT EXISTS agents (
    id TEXT PRIMARY KEY,
    name TEXT NOT NULL,
    agent_type TEXT DEFAULT 'unknown',
    first_seen TEXT NOT NULL,
    last_seen TEXT NOT NULL,
    sessions_count INTEGER DEFAULT 0
);

CREATE TABLE IF NOT EXISTS provenance (
    entry_id TEXT NOT NULL,
    source_agent TEXT NOT NULL,
    source_session TEXT NOT NULL,
    confidence REAL DEFAULT 0.5,
    created_at TEXT NOT NULL,
    PRIMARY KEY (entry_id, source_agent, source_session)
);

CREATE TABLE IF NOT EXISTS l2_cache (
    cache_key TEXT PRIMARY KEY,
    result_json TEXT NOT NULL,
    ttl_seconds INTEGER NOT NULL DEFAULT 86400,
    hit_count INTEGER DEFAULT 0,
    created_at TEXT NOT NULL,
    expires_at TEXT NOT NULL
);
```

## 6 Testing

Il progetto adotta una strategia di test a 3 livelli: unit test (logica interna pura, nessun MCP/subprocess), integration test (stesso processo, chiamate dirette a funzioni), e MCP smoke test (via mcp.ClientSession reale su STDIO subprocess). 61 test distribuiti su 10 file, zero mock – ChromaDB e SQLite reali in directory temporanee isolate per test (fixture pytest function-scoped con temp directory). Timing non usato come metrica (flaky su Windows); si usa cache.stats()['hit\_count'].

Tabella 3: Copertura dei test automatici.

| File              | N. | Scopo                                                  |
|-------------------|----|--------------------------------------------------------|
| test_embedding.py | 3  | Unit: dimensione 384, formato vettore, retrieval order |

| File               | N. | Scopo                                                           |
|--------------------|----|-----------------------------------------------------------------|
| test_memory.py     | 5  | Unit: ChromaDB add/query/delete/multi/integration               |
| test_cache.py      | 5  | Unit: L1 set/get/expiry/hit_count/clear                         |
| test_l2_cache.py   | 7  | Unit: L2 set/get/miss/expiry/hit_count/stats/clear/prune        |
| test_provenance.py | 7  | Unit: agent registry + provenance lifecycle                     |
| test_protocol.py   | 6  | Integration: FastMCP wiring, save→query, cache hit, L2 fallback |
| test_routing.py    | 9  | Unit: cosine similarity, exact/semantic/no match, warmup        |
| test_connectors.py | 8  | Unit: STDIO/HTTP properties, timeout, health, read-only         |
| test_graceful.py   | 6  | Graceful: backend failure, health cascade, retry, rate limit    |
| test_mcp_smoke.py  | 5  | Smoke: MCP ClientSession, full stack                            |

## 7 Decisioni Progettuali

Le seguenti decisioni progettuali sono state prese consapevolmente, con rationale esplicito. Non devono essere modificate senza discussione.

Tabella 4: Decisioni progettuali e rationale.

| Decisione                                | Razionale                                              |
|------------------------------------------|--------------------------------------------------------|
| FastMCP (non low-level Server)           | Sviluppo più rapido, codice più pulito                 |
| ChromaDB (non SQLite-VSS)                | Più maturo, documentazione migliore                    |
| Cache clear-all on save                  | Semplice, sicuro, sufficiente per MVP                  |
| Nessuna semantic cache                   | Due query simili possono essere operativamente diverse |
| all-MiniLM-L6-v2                         | 384d, leggero, sufficiente per MVP                     |
| Lazy backend connect                     | Nessun costo startup per backend idle                  |
| Routing: exact → semantic ( $\geq 0.5$ ) | Exact match tentato prima, fallback semantico          |
| Read-only prefix heuristic               | list_, get_, fetch_, read_, search_, find_...          |
| SQLite connection per-call               | Thread-safe, non ottimizzato per alta concorrenza      |
| Cosine similarity manuale (no numpy)     | Zero dipendenze esterne per routing                    |

## 8 Conclusioni e Roadmap

### 8.1 Roadmap Completata

#### Fase 1 - Core Memory

save/query/status con ChromaDB, sentence-transformers, TTLCache L1, FastMCP.

#### Fase 2 - Routing + Connectors

Router semantico + esatto, StdioConnector, HttpConnector, BackendRegistry, broadcast, read-only enforcement.

#### Fase 3 - Cross-Agent Memory

Provenance tracking (SQLite), L2 cache persistente, crossref, forget, parametri agent/session/confidence su save.

#### Fase 4 - Robustness

Logging strutturato, health check backend (healthy/unhealthy/dead), retry configurabili, rate limiting, non-blocking startup, WarmingUpError grace.

### 8.2 Roadmap Futura

- Fase 5: README completo, esempi reali, pubblicazione GitHub + PyPI (in corso).
- Chunking: gestione documenti multi-paragrafo con split per boundary semantico.

### 8.3 Metriche di Successo

Tabella 5: Metriche di successo.

| Metrica                     | Target                       |
|-----------------------------|------------------------------|
| Cache hit rate              | > 40% (MVP) / > 60% (target) |
| Latenza media query         | < 1000ms                     |
| Latenza cache hit           | < 50ms                       |
| Copertura routing semantico | >7%                          |
| Cross-agent knowledge reuse | 20                           |
| 61 test, 0 mock             | No, è un fatto.              |

MCP Agora non è un'invenzione. È un esercizio di ingegneria su un problema reale (memoria isolata tra agenti) con strumenti moderni (MCP, vector search, embeddings). Se risolve il problema di 4 agenti che non condividono memoria, ha già vinto.

## A config.yaml Completo

Il file config.yaml nella directory principale del progetto controlla la configurazione del server, dello storage, della cache, dell'embedding e dei backend.

Listing 2: File config.yaml completo.

```
agora:
  name: "Agora"
  version: "0.4.0"

storage:
  chroma_path: "~/.agora/chroma"
  db_path: "~/.agora/agora.db"

cache:
  l1_max_entries: 1000
  l1_ttl_seconds: 300
  l2_max_entries: 10000
  l2_ttl_seconds: 86400

embedding:
  provider: "sentence-transformers"
  model: "all-MiniLM-L6-v2"

backends:
  - name: "github"
    transport: "stdio"
    command: ["npx", "-y", "@modelcontextprotocol/server-github"]
    description: "GitHub API: issues, PRs, repos, code search"
    read_only: false
    timeout_seconds: 15
    max_retries: 3
    retry_delay: 1.0
    rate_limit_rps: 5
    env:
      GITHUB_TOKEN: "${GITHUB_TOKEN}"
  - name: "playwright"
    transport: "stdio"
    command: ["npx", "@playwright/mcp@latest"]
    description: "Browser automation: navigate, click, screenshot, forms"
    read_only: true
    timeout_seconds: 30
    max_retries: 2
    retry_delay: 0.5
    rate_limit_rps: 2
```

## B Struttura del Progetto

Listing 3: Struttura del progetto.

```
mcp-agora/
|-- pyproject.toml      # Dipendenze, build, entry point "agora"
|-- config.yaml        # Configurazione server + backend
|-- agora/
|   |-- main.py        # Entry point
|   |-- server.py      # FastMCP + 8 tools
|   |-- config.py      # YAML loader
|   |-- registry.py    # BackendRegistry
|   |-- connectors/
|       |-- base.py    # BackendConnector ABC
|       |-- stdio.py   # StdioConnector
|       +-- http.py    # HttpConnector
|   |-- routing/
|       +-- router.py  # Semantic + exact router
|   |-- embedding/
|       |-- base.py    # EmbeddingProvider ABC
|       +-- sentence.py # sentence-transformers
|   |-- memory/
|       +-- vector_store.py # ChromaDB wrapper
|   |-- cache/
|       |-- l1_memory.py # TTLCache
|       +-- l2_cache.py  # SQLite cache
|   +-- db/
|       +-- database.py  # SQLite schema + ops
|-- tests/
|   |-- test_embedding.py # 3 tests
|   |-- test_memory.py   # 5 tests
|   |-- test_cache.py    # 5 tests
|   |-- test_l2_cache.py # 7 tests
|   |-- test_provenance.py # 7 tests
|   |-- test_protocol.py # 6 tests
|   |-- test_routing.py  # 8 tests
|   |-- test_connectors.py # 9 tests
|   |-- test_mcp_smoke.py # 8 tests
|   +-- _echo_server.py
|-- docs/
|   +-- generate_pdf.py
|-- README.md
+-- LICENSE (MIT)
```

## C Cronologia Commit

Cronologia dei commit sul branch master del repository GitHub.

Listing 4: Commit history.

```
14dbdc5 Initial commit: MCP Agora Phase 1 complete
e209f40 Add MIT License
005590b Add agora usage instructions to AGENTS.md
5a64014 Phase 2: Routing + Backend Connectors
12e54ae docs: update README and AGENTS.md for Phase 2
```

```
0c36648 feat: HTTP connector, read_only, broadcast, timeout
b03abbe fix: ReadOnlyBlockedError public, parallel broadcast
dab75a2 feat: Phase 3 cross-agent memory
7f01873 docs: update ARCHITECTURE.md with Phase 2/3 details, add ruff linter
, generate PDF
c4aa01c feat: Phase 4 robustness - logging, health check, retry, rate limit,
graceful tests
40e86f6 fix: non-blocking startup + WarmingUpError grace
90765e8 chore: add *.log and chroma.sqlite3 to .gitignore
```